ntiny Core

Microarchitecture Reference

A bottom-up, module-by-module description of the ntiny RISC-V core,
its memory subsystem, and its behaviour under variable memory latency.

RV32IMACZicsr_Zifencei + Sv32 + PMP · 5-stage in-order

generated from `docs/microarch/`

Contents

1 Introduction

This document describes the **ntiny** core one module at a time, bottom-up: the memory leaves first, then the caches, the bus, the load/store and atomic units, the core control logic, and finally how they compose into `core_top` and the SoC. Each module gets a block diagram, a port summary, a plain-language walk-through, and—where it matters—a note on how it behaves when memory does *not* answer in a single cycle.

1.1 The pipeline, in one breath

ntiny is a classic in-order pipeline. Instructions flow through five stages:

`IF` (fetch) → `ID` (decode/align) → `IE` (execute) → `IMEM` (memory) → `IWB` (writeback).

A fetch front-end (PC logic, instruction cache, a fetch buffer, and a compressed aligner) feeds a decoder; the execute stage resolves branches and runs the ALU; the memory stage issues data accesses through a shared data bus; writeback commits results and CSR side effects. Forwarding paths and a central `hazard_unit` keep the pipeline correct without software-visible stalls.

1.2 The one idea that explains the hard bugs

Latency note. The original pipeline was *statically scheduled* around a memory that always answers in exactly one cycle. With single-cycle memory, "the load's data is on the bus exactly when the consumer needs it," so no dynamic interlock is required—correctness falls out of the fixed schedule. The moment memory latency becomes *variable* (a real cache miss, `+RAM_RANDOM_DELAY`, a future AXI/DRAM slave), that invariant breaks everywhere a memory result is produced or consumed: forwarding, branch/JALR resolution, the writeback commit gate, the fetch in-flight tracking, the bus response routing, and the cache itself. Each such site needs an explicit "data-not-ready-yet" handshake. This is why "just more wait time" is deceptively invasive—it is not one stall point, it is the removal of a global timing assumption. Throughout this document, **Latency notes** flag exactly where that assumption lived and what replaced it.

1.3 How to read the diagrams

Every diagram uses one consistent visual vocabulary so shapes always mean the same thing:

shape / colour	meaning
yellow rectangle	combinational logic (decode, mux net, ALU op)
blue rectangle	register / flip-flop (pipeline reg, latch, counter)
green stadium	a state machine (state register + transitions)
grey hexagon	multiplexer / select
purple cylinder	memory array (SRAM, cache tag/data arrays)
white box	an instantiated sub-module
pale boxes (in/out)	module ports, grouped left (inputs) / right (outputs)
solid arrow	data / datapath signal
dotted arrow	control / handshake / stall / freeze

Diagrams are deliberately *sparse*: node labels are short signal or block names, and all the detail lives in the prose of each section. The diagram sources are Mermaid (`docs/microarch/diagrams/*.mmd`); rebuild the figures and this PDF with `python3 docs/microarch/build.py`.

Part I

Privileged architecture

2 CSRs and privilege

Sources: ***design/core/csr_unit/****,* ***privilege_unit/****, and* ***core_pkg.sv****.*

ntiny implements three privilege levels—**M** (machine, `2'b11`), **S** (supervisor, `2'b01`), **U** (user, `2'b00`)—held in the `priv_level` register inside `csr_unit`. Reset enters M-mode. All architectural control state lives in the Control/Status Registers (CSRs); this section describes what is implemented and how privilege changes.

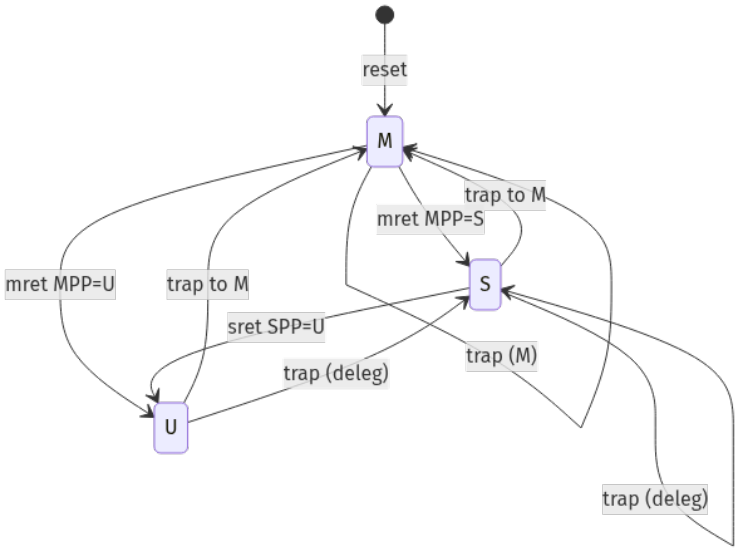


Figure 1: Privilege transitions. A trap raises privilege (to M, or to S when delegated); a return (`mret`/`sret`) lowers it to the saved previous privilege (`MPP`/`SPP`). Reset starts in M.

2.1 Two units, two jobs

`privilege_unit` is a small *combinational* checker at the ID stage: it decides whether an instruction is legal at the current privilege. It flags illegal CSR accesses (a CSR's address bits `[9:8]` encode its minimum privilege; access is illegal when `priv_level` is lower), illegal `mret` (needs M) / `sret` (needs S), and—when `mstatus.TVM=1` in S-mode—traps on `satp` writes and `sfence.vma`. `csr_unit` is the *state*: it stores every CSR, performs reads/writes with WARL masking, applies the atomic trap-entry / `xret` side effects, and drives `priv_level`, `satp`, the delegation registers, and the interrupt pending/enable views out to the rest of the core.

2.2 Implemented CSRs (overview)

group	registers	notes
M trap setup	`mstatus(h)`, `misa`, `medeleg`, `mideleg`, `mie`, `mtvec`	`misa` reads RV32 + the implemented extensions; deleg selects M-vs-S
M trap handling	`mscratch`, `mepc`, `mcause`, `mtval`, `mip`	`mepc/mcause/mtval` auto-written on an M-trap
M counters	`mcycle(h)`, `minstret(h)`, `mcountinhibit`, `mcounteren`	HPM counters 3–31 are WARL read-zero
M env / Sstc	`menvcfg(h)` (STCE, ADUE), `stimecmp(h)`	STCE enables the Sstc S-timer; ADUE enables HW A/D updates
PMP	`pmpcfg0..3`, `pmpaddr0..15`	16 regions; locked entries (L bit) are write-protected
S mode	`sstatus`, `sie`, `stvec`, `scounteren`, `sscratch`, `sepc`, `scause`, `stval`, `sip`, `satp`, `senvcfg`	`sstatus/sie/sip` are filtered views of the M registers
U mode	`fflags`, `frm`, `fcsr` (FPU), `seed` (Zkr), `cycle/time/instret(h)`	user counters are read-only, gated by `[m/s]counteren`
Debug	`tselect/tdata1..3/tinfo/tcontrol`	trigger module present but no real trigger slot (WARL)
Hart info	`mhartid`=0, `mvendorid/marchid/mimpid/mconfigptr`=0	single hart

2.3 `mstatus` / `sstatus`

`mstatus` holds the privileged mode/interrupt state. The live fields are: `MIE/SIE` (global interrupt enable per level), `MPIE/SPIE` (the pre-trap enable, restored on return), `MPP[1:0]/SPP` (the pre-trap privilege, restored on return—`SPP` is a single bit since S can only have come from U or S), `MPRV` (data accesses use `MPP`'s privilege), `SUM` (S may touch U pages), `MXR` (loads may read execute-only pages), `TVM/TW/TSR` (trap virtual-memory ops / WFI / `sret` in S-mode), and `FS[1:0]/SD` for the FPU. `sstatus` is the same register read and written through a mask that exposes only the S-visible bits (`SIE`, `SPIE`, `SPP`, `FS`, `SUM`, `MXR`, `SD`); writes to any other bit through `sstatus` are dropped.

2.4 The CSR read/write datapath

CSR instructions execute at the IE stage. A read is a combinational mux over the CSR address, with WARL read-masks applied (e.g. `sstatus` returns `mstatus` & read-mask; HPM and unimplemented CSRs return 0). A write (`csrrw`), set (`csrrs`), or clear (`csrrc`) only changes the bits that the register's write-mask (WMASK) allows. Read-only and reserved bits never change. An access to an unimplemented (non-HPM) CSR raises illegal-instruction. Notable side effects: writing `satp` re-points the MMU; writing `mstatus/mie/mip` changes interrupt behaviour the same cycle; `sie/sip` only touch the S-delegated bits of the underlying `mie/mip`.

2.5 Interrupt-pending hardware vs. software bits

`mip` mixes hardware-driven and software-driven bits. The machine bits `MTIP/MSIP/MEIP` are refreshed every cycle from the uncore (`interrupt_src` from CLINT/PLIC) and are read-only to software. The supervisor bits `STIP/SSIP/SEIP` are software-writable by M-mode (to inject S-interrupts), *plus* a hardware contribution: `SEIP` ORs in the PLIC S-context line, and `STIP` is driven in hardware by Sstc when `menvcfg.STCE=1` (`mtime` $\geq$ `stimecmp`). When Sstc is enabled,

software writes to `STIP` are ignored—the timer is owned by the comparator.

2.6 Trap entry and return (CSR view)

On a trap, `csr_unit` latches `[m/s]epc`, `[m/s]cause`, `[m/s]tval`, updates `mstatus` (push old privilege into `xPP`, old enable into `xPIE`, clear `xIE`), and sets `priv_level`. The crucial sequencing detail—covered in §3.7—is that an in-flight `csr` write on the trapped instruction is applied *first*, then the trap-entry overrides layer on top, because the saved `epc` points past that instruction. On `mret/sret` the inverse happens: `xIE`←`xPIE`, `xPIE`=1, `xPP`=U, and `priv_level` is restored from `xPP`; `MPRV` is also cleared if returning below M.

3 Traps, exceptions and interrupts

Sources: `design/core/interrupt/`, `trap_sequencer/`, `wb_trap_unit/`, `csr_unit/`, *and their wiring in* `core_top.sv`.

A *trap* is any forced, non-sequential change of control flow into a handler: a synchronous *exception* caused by the instruction itself (illegal opcode, `ecall`, a page/access fault, a misaligned access), or an asynchronous *interrupt* from outside the pipeline (timer, software, or external). This section explains how ntiny detects, prioritises, commits, and returns from traps.

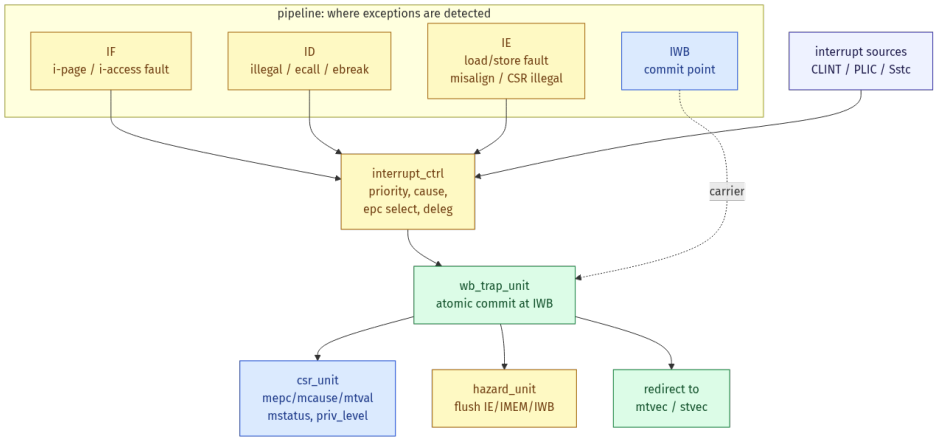


Figure 2: Trap flow. Exceptions are detected in the stage that can see them (top); interrupts arrive from CLINT/PLIC/Sstc. `interrupt_ctrl` picks the winner, its cause, the saved PC (epc) and whether it delegates to S-mode; `wb_trap_unit` commits it atomically at writeback, which drives the CSR updates, the pipeline flush, and the redirect to the handler.

3.1 The big idea: traps commit at writeback (IWB)

ntiny resolves every trap, interrupt and return (`mret`/`sret`/`dret`) **atomically at the IWB stage** (`wb_trap_unit`). An instruction is tagged during decode/execute with a `wb_event` (`WB_TRAP`, `WB_XRET`, `WB_DRET`); when that tagged "carrier" reaches IWB, the trap fires. Committing at one fixed point—rather than wherever the fault was first noticed—is what makes the privileged state consistent: it removes the old races between an `mret` and the `csrrw mepc` just ahead of it, and between a page-table walk and the load that triggered it. When a trap fires, `wb_trap_unit` asserts `kill_iwb`: the carrier does *not* write its register or commit its store, and a four-stage flush (IE/IMEM/IWB plus the fetch redirect) kills every younger wrong-path instruction.

3.2 Exception sources

Each exception is detected in the earliest stage that has the information, then carried to the commit point. The saved PC (`epc`) is the faulting instruction for exceptions (so the handler can fix up and retry), and `mtval`/`stval` carries the offending address or instruction word.

exception	cause	detected in	tval
Instruction access fault (PMP)	1	IF (PMP / PTW)	faulting VA (or straddled half)
Illegal instruction	2	ID, or IE (bad CSR)	instruction word
Breakpoint (`ebreak`)	3	ID	0
Load address misaligned	4	IE	address
Load access fault (PMP)	5	IE	address
Store/AMO misaligned	6	IE	address
Store/AMO access fault (PMP)	7	IE	address
ECALL from U / S / M	8/9/11	ID	0
Instruction page fault	12	IF (ITLB / PTW)	faulting VA
Load page fault	13	IE (DTLB / PTW)	address
Store/AMO page fault	15	IE (DTLB / PTW)	address

Two implementation details worth knowing. First, instruction-side faults (causes 1 and 12) take *two* paths: a combinational one that rides through the `fetch_buffer` on the same entry as the faulting word (so it fires when that word reaches the buffer head), and a direct "PTW-completion" path for faults discovered by the page-table walker (which has no live fetch request to ride). Second, the data-side fault's *is-store* flag is registered (`trap_sequencer`) because the trap can fire a cycle or two after the IE stage has already flushed and dropped the live store signal.

3.3 Interrupt sources

Three interrupt lines arrive from the uncore and become pending bits in `mip`: machine timer (`MTIP`, from the CLINT `mtime` $\geq$ `mtimecmp` compare), machine software (`MSIP`, CLINT), and machine external (`MEIP`, PLIC). Their S-mode counterparts (`STIP`/`SSIP`/`SEIP`) are either injected by M-mode software via `mip`, sourced from the PLIC's S-mode context, or—for the timer—generated in hardware by the **Sstc** extension (`stimecmp` compared against `mtime` when `menvcfg.STCE=1`). An interrupt is *taken* only when its pending bit and its enable bit (`mie`/`sie`) are both set *and* the global enable for the current privilege allows it: in M-mode that is `mstatus.MIE`; below M-mode, M interrupts are always enabled. Delegation (`mideleg`) sends a delegated interrupt to S-mode instead of M-mode when not already in M-mode.

Latency note. Interrupts are held off until the pipeline actually has an in-flight instruction to pin them to (`pc_id` or `pc_ie` non-zero). If a timer fires into an empty/draining pipeline, naively using the fetch-stage PC as `epc` would set the return address *ahead* of instructions still in flight, double-executing or skipping them on `sret`. Deferring the interrupt one cycle until the pipeline is populated avoids that.

3.4 Priority

Synchronous exceptions outrank interrupts on the same instruction, and older (deeper-in-the-pipeline) exceptions outrank younger ones. The order, highest first: IE-stage data faults (PMP access, then page fault, then misalign, then illegal-CSR) $\rightarrow$ IF/ID-stage faults (instruction access, instruction page, illegal, `ecall`, `ebreak`) $\rightarrow$ interrupts (external, then software, then timer; M-mode before S-mode). At the IWB commit, a debug entry outranks everything, then an interrupt preempts the carrier, then a synchronous trap, then `xret`.

3.5 Choosing the saved PC for an interrupt

The trickiest part of the trap logic—and historically the most bug-prone—is picking `epc` for an *asynchronous* interrupt, because the "current" instruction depends on what the pipeline is

doing that cycle. `interrupt_ctrl` selects `pc_for_async` with a priority fallback:

1. a branch/jump is committing this cycle → its architectural next-PC (`branch_recovery_target`), which collapses all predicted×actual direction cases into the correct resume point;
2. an IE-stage op has not committed (AMO in progress, data MMU stall) → `pc_ie` (re-execute it after `sret`);
3. `pc_id` is non-zero → `pc_id` (the next instruction to enter IE);
4. `pc_ie` is non-zero → `pc_ie + insn_len` (length-aware: 2 for compressed, 4 otherwise);
5. otherwise the fetch-stage PC (only when the pipeline is empty, e.g. just after an `xret`).

Several past Linux-boot hangs were exactly a wrong choice here (e.g. a timer firing the same cycle as a taken-and-correctly-predicted branch, where the naive fall-through PC is the wrong resume target).

3.6 Trap entry: what hardware writes

When the trap commits, `csr_unit` performs, in one cycle, all of:

- `mepc`/`sepc` ← the selected `epc`;
- `mcause`/`scause` ← the cause (interrupt bit set for asynchronous);
- `mtval`/`stval` ← the trap value (0 for interrupts and `ebreak`);
- `mstatus`: for an M-trap, `MPP`←old privilege, `MPIE`←`MIE`, `MIE`←0 (the S-trap form uses `SPP`/`SPIE`/`SIE`, with `SPP` a single bit);
- `priv_level` ← M (`2'b11`) or S (`2'b01`) per delegation.

The handler address is `mtvec`/`stvec` with its low two bits stripped; in vectored mode (base[0]=1) an *interrupt* targets base + 4×cause, while exceptions always go to the base.

3.7 The carrier still commits its CSR write

Latency note. Subtle but important: when an asynchronous interrupt preempts a CSR instruction (say `csrsi sstatus,2` that sets `SIE`), that CSR write must still take effect, because the chosen `epc` points *past* it—the architecture has logically retired it. `csr_unit` therefore computes the new `mstatus` by first applying the in-flight CSR write and *then* layering the trap-entry overrides (`MPIE`←post-write `MIE`, etc.) on top. A real Linux scheduler hang was caused by the earlier code suppressing the carrier's CSR write on a coincident trap: `SIE` stayed 0, so the kernel resumed with interrupts disabled and warned in `do_idle()`.

3.8 Trap return: `mret` / `sret` / `dret`

A return is decoded, tagged `WB_XRET` (or `WB_DRET`), and commits at IWB. `mret` restores `MIE`←`MPIE`, sets `MPIE`=1, `MPP`=U, clears `MPRV` if returning below M, and jumps to `mepc`; `priv_level`←`MPP`. `sret` is the analogous S-mode form (`SIE`←`SPIE`, `SPIE`=1, `SPP`=U, jump to `sepc`). `dret` restores `dpc` and leaves debug mode without a privilege change.

Because a return changes privilege, the fetch of the first handler-exit instruction must use the *new* privilege for translation. A small fetch FSM (`xret_fetch_ctrl`) drains speculative wrong-privilege fetches, waits one cycle for `priv_level` to settle, then re-fetches from the return

target. A companion guard in `trap_sequencer` suppresses stale instruction faults from the *old* privilege's in-flight fetch while the `sret` CSR side effects land—otherwise a cold-ITLB page walk on the `sret` target could, several cycles later, overwrite `sepc`/`scause` with a garbage fault.

3.9 Interaction with the stall chain

Latency note. Traps share the pipeline with the variable-latency hazards from the rest of this document. Two cases matter. (1) An interrupt coincident with a *stalled load* must save `pc_ie` (case 2 above) so the load re-executes after `sret`; if its data happens to arrive the same cycle, the IE flush suppresses the (now wrong-path) register write, and the post-`sret` re-execution re-reads memory correctly. (2) A speculative fetch past an unconditional jump can sit in a no-execute PMP region; its access fault must be *squashed* when an older redirect (branch/`xret`/debug) is firing, so a wrong-path fault never reaches the handler. Both are handled by gating the fault on the live redirect/flush signals rather than letting it fire blindly.

4 MMU (Sv32) and PMP

Sources: `design/core/mmu/src/mmu_sv32.sv`, `pmp_checker.sv`, *and the PTW arbitration in* `core_top.sv`.

When `satp.MODE`=1 the core runs in **Sv32**: 32-bit virtual addresses translate through a two-level page table to (up to 34-bit) physical addresses, with 4 KB pages and 4 MB superpages. M-mode never translates; S/U-mode do (and data accesses may borrow `MPP`'s privilege when `MPRV`=1). Independent of translation, every physical access is checked by PMP.

4.1 TLBs and the page-table walker

Translation is cached in two 8-entry, fully-associative TLBs (one instruction, one data), tagged by VPN + ASID with a per-entry "megapage" flag and the PTE permission bits. A hit translates in zero extra cycles; a miss starts the page-table walker (PTW).

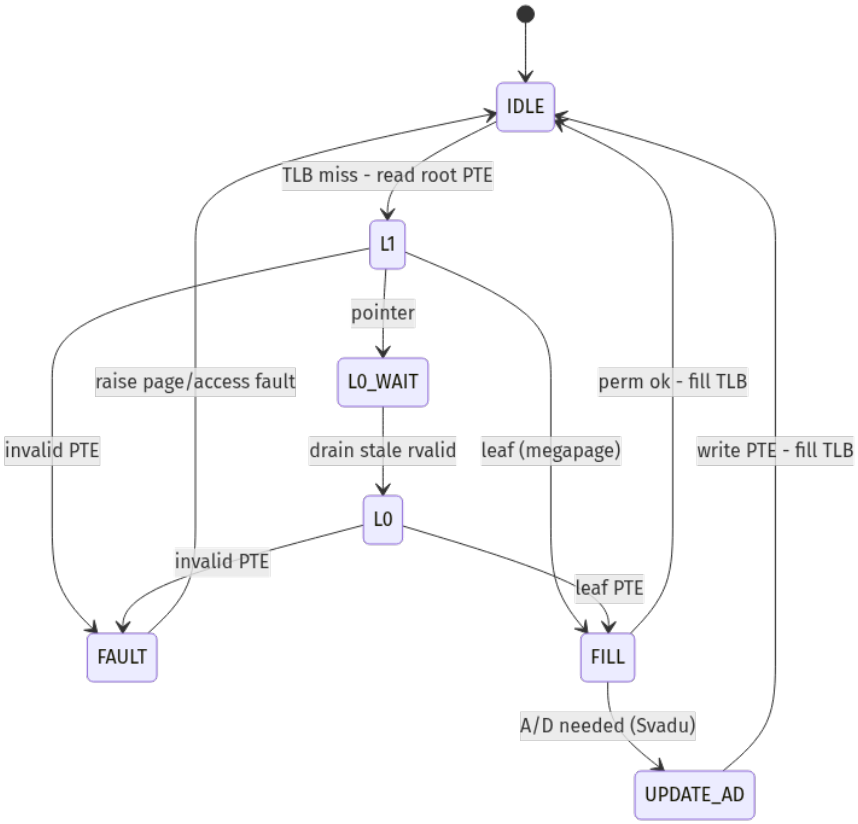


Figure 3: Sv32 page-table walker. It reads the root PTE (`L1`), then—if that is a pointer—the leaf PTE (`L0`); a leaf with bad permissions or an invalid PTE faults; a leaf needing the A/D bits set is updated in place (Svadu) before the TLB is filled. `L0_WAIT` drains a stale response between the two reads.

The PTW is a *master on the data bus*: it issues PTE reads (and Svadu A/D writeback writes) through `dmem_arb`, which arbitrates the PTW against the LSU and the AMO unit. While the PTW is active it asserts `ptw_active`, which stalls both the instruction and data sides (`mmu_i_stall` / `mmu_d_stall`) so nothing consumes a half-translated address. A megapage is a leaf found at L1; its `PPN[0]` must be zero (else it faults as misaligned), and the page's low VPN supplies the corresponding PA bits.

Latency note. The PTW is the original variable-latency master: a walk is two dependent memory reads whose timing the pipeline cannot predict. Its `ptw_active` stall, the `LO_WAIT` drain state, and the way it latches the initiating privilege/SUM/MXR at walk start are all there because the walk spans many cycles during which the rest of the machine keeps moving. This is the same "a memory access is not single-cycle" problem the cache and LSU sections describe, met first by the MMU.

4.2 Permissions, SUM, MXR, A/D

A translation is permitted only if the leaf PTE's permission bits allow the access at the effective privilege. Key rules as implemented: instruction fetch requires `X` and (for U pages from S-mode) is *always* forbidden—even with `SUM`; a load requires `R`, or `X` when `MXR`=1; a store requires `W` *and* the `D` (dirty) bit; S-mode access to a U page needs `SUM`=1. The accessed (`A`) and dirty (`D`) bits must be set before use: if `menvcfg.ADUE`=1 the PTW sets them in hardware (the `UPDATE_AD` state writes the PTE back), otherwise the missing bit raises a page fault for software to fix. `sfence.vma` flushes both TLBs and aborts any in-flight walk (so a stale walk cannot re-fill a just-cleared entry).

4.3 Faults

A permission failure or invalid PTE becomes the matching page fault—cause 12 (instruction), 13 (load), 15 (store). These are routed into the trap path exactly like other exceptions (§2). The is-load-vs-store distinction is registered so it survives the cycle or two until the trap commits.

4.4 PMP

PMP guards physical addresses with 16 regions, each a config byte (`R/W/X`, an address-match mode, and a lock bit `L`) plus an address register. Match modes are `OFF`, `TOR` (top-of-range, vs. the previous entry's address), `NA4` (one aligned word), and `NAPOT` (a naturally-aligned power-of-two region encoded in the address bits). A priority encoder scans entries low-index-first; the first match decides. M-mode bypasses PMP unless a *locked* entry explicitly denies it; for S/U the default (no match) is deny. A PMP violation becomes an access fault—cause 1 (instruction), 5 (load), 7 (store). PMP is checked on the *translated* physical address, so PTW reads are themselves PMP-checked at the privilege that started the walk; a PMP-denied PTW read turns the walk into a fault rather than a page fault.

Part II

Memory subsystem

5 Instruction cache (icache)

Source: `design/memory/src/icache.sv`

5.1 Purpose

The `icache` sits between the fetch front-end and instruction-side RAM (port A). It answers instruction fetches from a small on-chip array when it can (a *hit*), and fetches a missing word from RAM (a *miss*) otherwise. It is a **single-outstanding stalling slave**: it can hold the fetch unit off (`cpu_ready`=0) while it services a miss, and it tolerates RAM that takes an arbitrary number of cycles to answer.

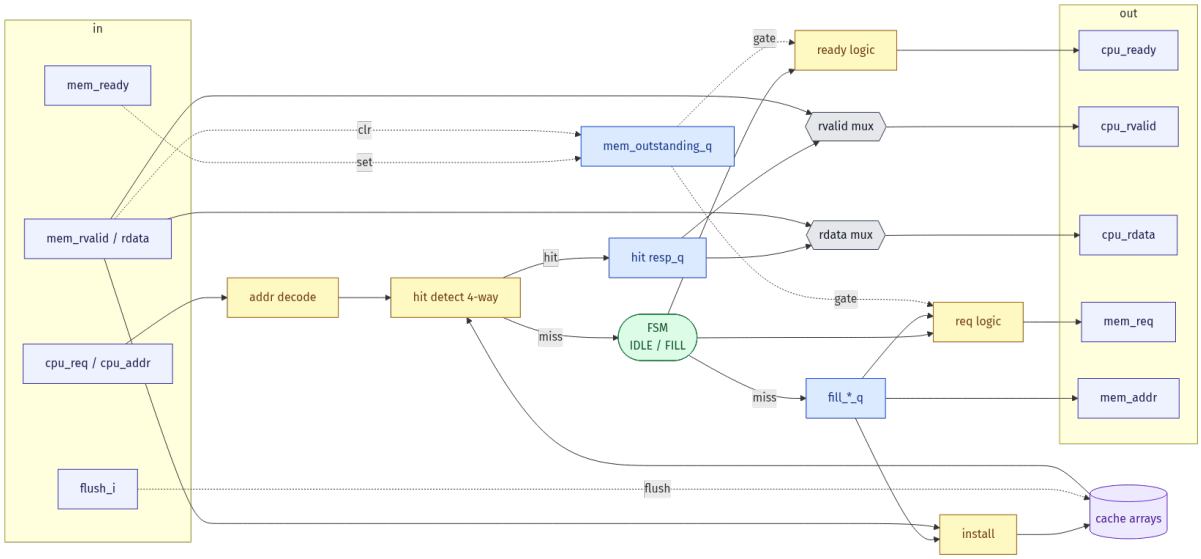


Figure 4: `icache` datapath. Left: CPU/fetch and RAM inputs. Right: CPU and RAM outputs. The FSM, the captured fill registers, and the `mem_outstanding` tracker are what make it latency-tolerant.

5.2 Geometry

4 KB, 4-way set-associative, 32-byte lines (8 words per line), 32 sets. An address splits as `tag[31:10]`, `index[9:5]`, `word_off[4:2]`, byte offset `[1:0]` (ignored—fetches are word-aligned). Each line carries a `line_valid` bit *and* per-word `word_valid` bits, so a single-word fill installs only the word that was returned; a hit needs both bits set. Replacement is round-robin per set (`repl_ptr`).

5.3 Ports

port	dir	meaning
cpu_req_i / cpu_addr_i	in	fetch request + word address
cpu_rdata_o	out	fetched instruction word
cpu_rvalid_o	out	response valid (one pulse per accepted req)
cpu_ready_o	out	1 = can accept a new request this cycle
flush_i	in	FENCE.I: invalidate all lines
mem_req_o / mem_addr_o	out	RAM read request to port A
mem_rdata_i / mem_rvalid_i	in	RAM read response
mem_ready_i	in	RAM accepts a request this cycle

5.4 How a request is served

Address decode and hit test are combinational on cpu_addr_i: the four ways are checked in parallel for (line_valid AND tag match AND word_valid); hit is their OR and hit_way_id selects the matching way.

On a hit, the cache registers a one-cycle response: resp_valid_q pulses next cycle and resp_data_q holds the selected word. cpu_ready stays high, so hits accept back-to-back at one fetch per cycle—the same behaviour fetch always saw.

On a miss, the FSM (§5.6) leaves S_IDLE for S_FILL, drops cpu_ready to 0 (the fetch unit now *holds* its address and request stable), captures the request into fill_tag/index/word/addr_q with fill_live_q=1, and drives mem_req to RAM. When the RAM read returns (mem_rvalid), the word is installed into the array and forwarded to the CPU as cpu_rvalid with mem_rdata; the FSM returns to S_IDLE.

Install (install_tag_match) is recomputed against the *current* array state at write time, so back-to-back fills of consecutive words in the same line land in the way the previous fill just allocated instead of picking a fresh way (this avoids a two-ways-same-tag corruption seen earlier in Linux init).

FENCE.I (flush_i) invalidates every line in one cycle and blocks new accepts; a fill already in flight is abandoned but its RAM read is still drained (below).

5.5 Why it is latency-tolerant

Latency note. The previous icache was *transparent*: cpu_ready=mem_ready, mem_addr=cpu_addr combinationally, and the response was derived from the *live* cpu_addr. That silently assumes RAM returns data exactly one cycle after the request. Under variable latency the fetch unit has already advanced cpu_addr to the next word by the time the data arrives, so the transparent cache returned the *wrong-offset* word (e.g. it answered a fetch of 0x..04 with the word at 0x..08), which double-executed branch targets and corrupted beq/bne/cj results. The fix is the structure in Fig. 4: cpu_ready drops during a fill so the master holds; the fill address is *captured* and used for the RAM request, the install slot, and the response; and mem_outstanding_q keeps the RAM port strictly single-outstanding (so an abandoned FENCE.I fill's late response is swallowed before a new fill can issue). Under always-ready RAM mem_outstanding clears in one cycle, so a miss simply costs one stall cycle—baseline behaviour is unchanged.

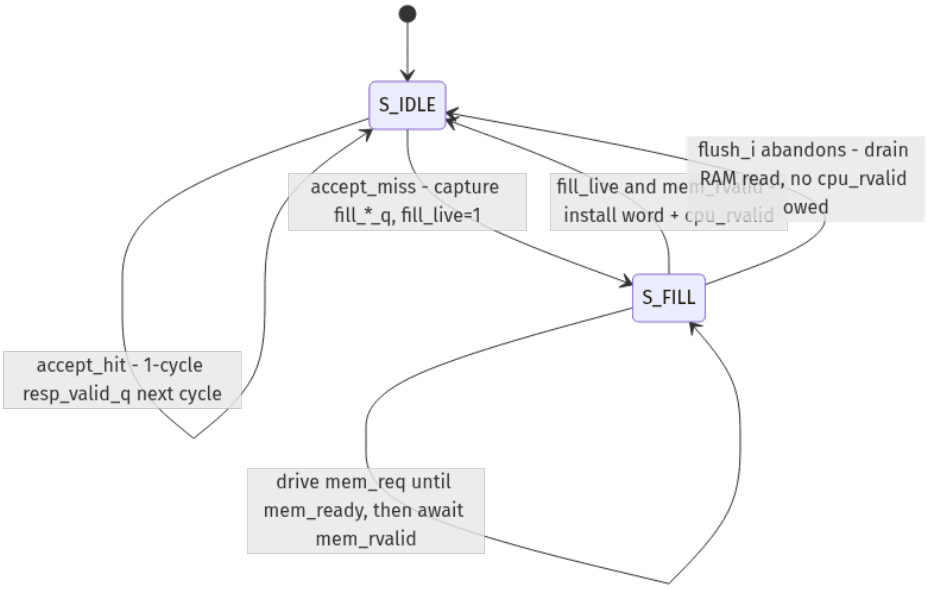


Figure 5: `icache` fill FSM. A hit is served from `S_IDLE` in one cycle; a miss enters `S_FILL`, holds the master off, issues the captured RAM read, and waits for `mem_rvalid`.

5.6 Fill FSM

from	to	when / action
`S_IDLE`	`S_IDLE`	accept_hit: register `resp_valid_q` for next cycle
`S_IDLE`	`S_FILL`	accept_miss: capture `fill_*_q`, set `fill_live`
`S_FILL`	`S_FILL`	drive `mem_req` until `mem_ready` accepts, then await `mem_rvalid`
`S_FILL`	`S_IDLE`	`fill_live` & `mem_rvalid`: install word, pulse `cpu_rvalid`
`S_FILL`	`S_IDLE`	`flush_i` abandons the fill; RAM read drained, no `cpu_rvalid` owed

6 Load/Store unit (core2avl)

Source: `design/core/avalon_master/src/core2avl.sv`

6.1 Purpose

`core2avl` is the CPU's load/store unit (LSU). It turns a decoded memory operation (a load or store of a byte/half/word at some address) into bus transactions, and formats returning read data (sign- or zero-extended, shifted to the right byte lanes). A naturally-aligned access is one bus beat; a half/word that straddles a 32-bit word boundary is split into two aligned beats by a small FSM.

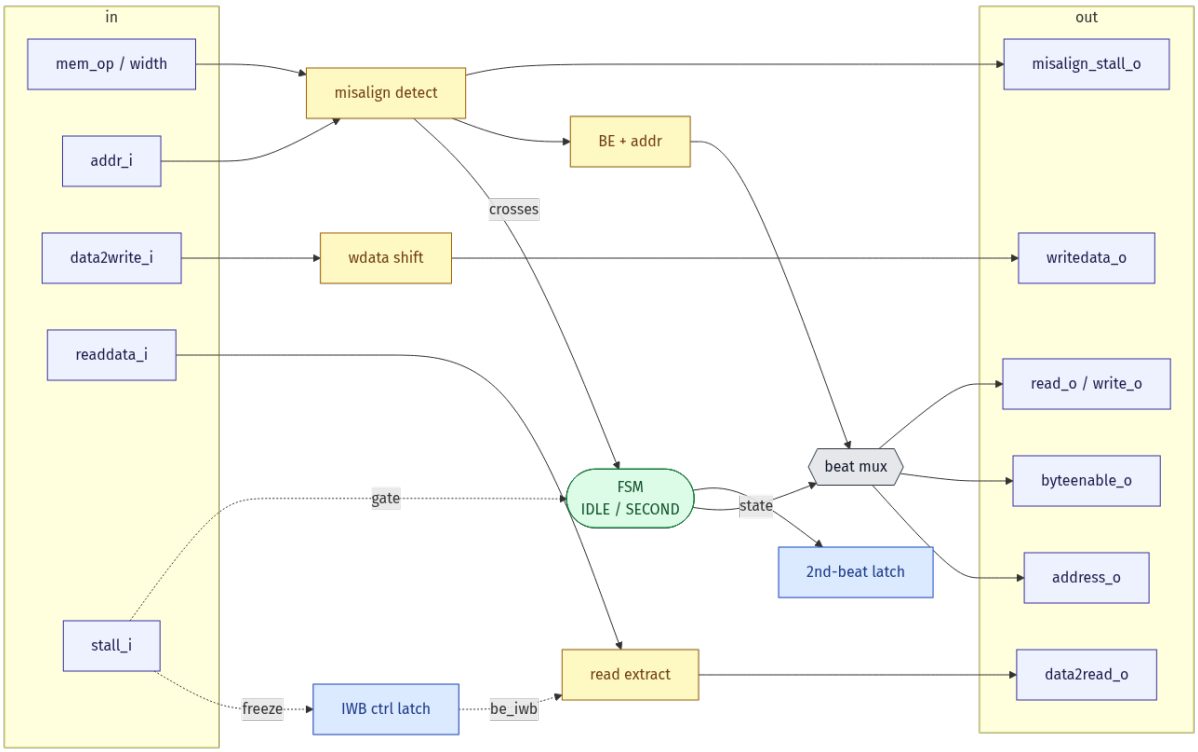


Figure 6: `core2avl` datapath. Address/op enter on the left; bus request signals and the formatted read result leave on the right. The IWB ctrl latch (blue) holds the formatting controls across a multi-cycle load.

6.2 Ports

port	dir	meaning
mem_op / load_store_width / mem_unsigned	in	op kind, size, signedness
addr_i	in	access address (from the ALU)
data2write_i	in	store data
readdata_i	in	read data from the bus
stall_i	in	freeze the IWB control latch + FSM (= iwb_stall)
address_o / byteenable_o / writedata_o	out	bus request fields
read_o / write_o	out	bus request strobes
data2read_o	out	formatted load result (to IWB)
misalign_stall_o	out	1 during the first beat of a crossing access

6.3 How it works

Misalign detect computes, combinationally from the width and addr[1:0], whether the access crosses a word boundary. If not, the access is a single aligned beat: address_o=addr_i, byteenable_o is the byte-lane mask for that size/offset, and writedata_o is the store data shifted into the correct lanes.

Crossing accesses use the IDLE → SECOND FSM. The first beat reads/writes the low part at the aligned address; the operands (addr_q, wdata_q, width_q, op_q) and the first read word are latched; the next cycle issues the second beat at the next word, and the two halves are recombined on the read path. misalign_stall_o holds the pipeline for the extra beat.

Read formatting (read extract) selects the right bytes from readdata_i using a small *IWB control latch*—be_iwb, mode_iwb, mem_unsigned_iwb, byt_iwb—that was captured when the access was issued, then applies sign/zero extension to produce data2read_o.

6.4 The IWB control latch and variable latency

Latency note. read extract formats *live* readdata_i using the *registered* controls in the IWB ctrl latch. Those controls are clocked by stall_i. Originally stall_i was tied to 0, so the latch updated every cycle. With single-cycle memory that is fine: the read data arrives the one cycle the controls are still valid. Under variable latency the load's data arrives several cycles later—by which point the *next* instruction (often a non-load) has overwritten be_iwb to "no byte lanes," so the returning word formats to 0. The fix is to drive stall_i=iwb_stall: the latch (and the FSM) freeze while the load waits, so be_iwb survives until readdata_i is valid. Because iwb_stall is permanently 0 under always-ready RAM, baseline behaviour is unchanged.

7 Data cache (dcache)

Source: ***design/memory/src/dcache.sv***

This section is deliberately general. The ***dcache*** *is slated to be replaced by the generated* ***iob-cache*** *(AXI back-end), so what matters here is the* contract *it presents—geometry, the master/slave handshake, and the no-stale-data invariant—not its bit-level internals.*

7.1 What it is today

The `dcache` is the data-side L1 of the bus revamp's Phase 2a. It is **4-way set-associative**, **32-byte lines**, **32 sets**, **4 KB** total (`dcache.sv:48--58`), with a VIPT geometry mirroring the icache. But it is not yet a real caching cache: it is **transparent write-through**. Every CPU request is forwarded to backing memory, and `cpu_rdata_o` is wired straight from the memory response (`dcache.sv:93--104`). The tag/data arrays are filled opportunistically so a later phase (2c) can flip on read-from-cache, but right now they never *serve* a load—they only shadow memory.

The reason for this half-step is the invariant in the header comment: the pipeline must never see stale cached data on a write→read hazard. Wiring reads through to memory guarantees that for free while the geometry and fill machinery are brought up and verified.

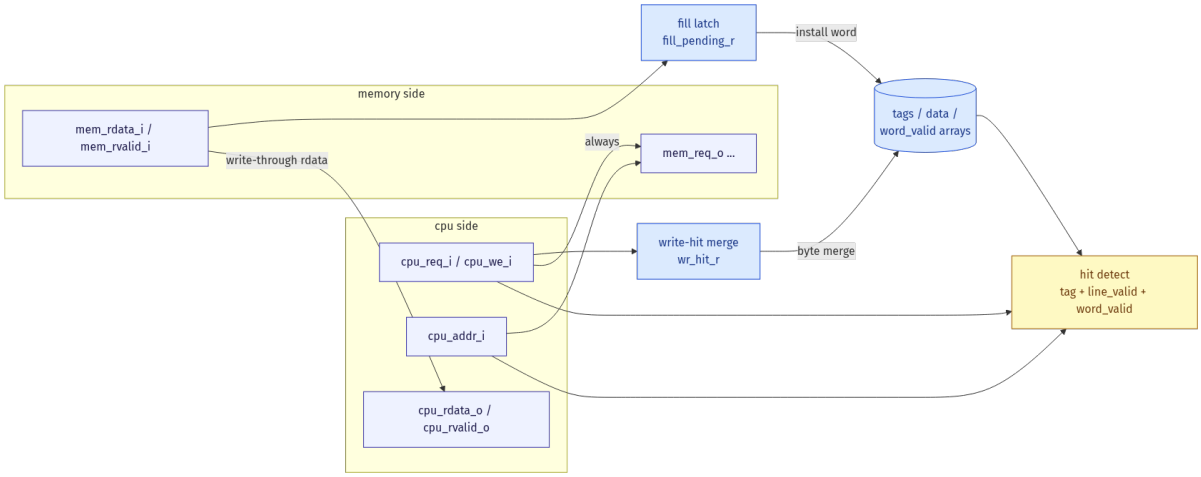


Figure 7: `dcache` block view. The CPU request always forwards to memory (`mem_req_o`); the read result returns straight from `mem_rdata_i` (write-through). The tag/data arrays (blue) are filled and write-merged in the background so a future phase can read from them.

7.2 Geometry and arrays

A 32-bit address splits as tag `[31:10]` / index `[9:5]` / word-offset `[4:2]` / byte-offset `[1:0]` (`dcache.sv:55--58`). Storage is four parallel arrays indexed by `[way][set]`: `line_valid`, `tags`, `data`, and a *per-word* `word_valid[way][set][word]` (`dcache.sv:62--65`). The per-word valid bits let a transparent fill install *one* word per memory response without wrongly marking the rest of the line present. Replacement is round-robin per set (`repl_ptr[set]`). There are no dirty bits—write-through means memory is always current.

A hit is the parallel AND across ways of `line_valid & (tags==addr_tag) & word_valid[...word]` (`dcache.sv:74--82`); `hit` is the OR of the four. Even on a hit the request still goes to memory today—the hit signal only gates the background bookkeeping, not the data path.

7.3 Handshake: master and slave sides

The CPU-facing (slave) port is the now-familiar latency-tolerant handshake: `cpu_req_i`/`cpu_we_i`/`cpu_addr_i`/`cpu_be_i`/`cpu_wdata_i` in, `cpu_ready_o` to accept, and `cpu_rdata_o` + `cpu_rvalid_o` as a separate response beat. The memory-facing (master) port mirrors it: `mem_req_o...`, `mem_ready_i`, `mem_rdata_i` + `mem_rvalid_i`. Because the cache is transparent, the two ports are essentially tied together: `mem_req_o`=`cpu_req_i` and `cpu_ready_o`=`mem_ready_i`.

7.4 The latency contract (the part iob-cache must keep)

Latency note. The `dcache` *itself* produces no pipeline stall. It does not own a busy signal; it just forwards `cpu_ready_o`=`mem_ready_i` and lets the response beat (`cpu_rvalid_o`) arrive whenever the slave returns it. All the variable-latency handling lives *outside* the cache—in `core2avl` and the load-pending tracking in `core_top`, and in `soc_top`'s `rd_inflight_q` routing (next section). This is exactly the property that makes `iob-cache` a near drop-in: its `iob_s` front-end is the same "request accepted by `ready`, data returns later on a valid" split, so the core-side stall fixes that were done for variable latency are the prerequisite, not wasted work. The fill machinery here uses a single-outstanding assumption (ready drops during a miss so a second read cannot be issued, `dcache.sv` + `soc_top`); any replacement must preserve at least that contract.

7.5 Flush

`FENCE.I` drives `flush_i`, which clears every `line_valid` and `word_valid` bit in one cycle (`dcache.sv:173--181`). The same wire flushes the icache. Plain `FENCE` is a no-op for a write-through cache over always-current memory, and `sfence.vma` is handled in the MMU, not here.

8 Data-bus arbiter (dmem_arb)

Source: *design/interconnect/src/dmem_arb.sv.*

Three masters want the one data bus: the page-table walker (PTW), the atomic unit (AMO), and the normal load/store unit (core2avl, called c2a here). dmem_arb decides who gets the bus and routes the reply back to the right master.

8.1 Who wins

The priority is fixed: **PTW** > **AMO** > **c2a**. The grants are combinational:

- ptw_grant_o: PTW wins whenever ptw_active_i is set;
- amo_grant_o: AMO wins when amo_active_i is set, PTW is idle, and suppress_data is low;
- c2a_grant_o: c2a wins when neither PTW nor AMO is active and suppress_data is low.

suppress_data is the OR of mmu_d_access_fault_i, mmu_d_fault_i, and d_xlat_pending_i. It blocks AMO and c2a (but not PTW) while the MMU is translating or a data fault is firing, so no access commits with a half-translated or faulting address. The chosen master drives the shared slave port bus (bus.addr, bus.we, bus.be, bus.wdata, bus.req). AMO and c2a share the MMU-translated address data_paddr_i; PTW uses its own ptw_addr_i.

8.2 Routing the reply

A read reply comes back later, so the arbiter must remember who asked. pending_rd_master_q (one of M_NONE/M_PTW/M_AMO/M_C2A) records the owner of the outstanding read. It is set when a read is accepted (read_accepted = bus.req & ~bus.we & bus.ready) and cleared when the reply arrives (bus.rvalid). Then each master only sees its own reply: c2a_rvalid_o = (pending_rd_master_q == M_C2A) & bus.rvalid, and the same for PTW and AMO. *_ready_o = *_grant_o & bus.ready.

Latency note. This ownership flop is what makes the bus safe under variable latency. With single-cycle memory the reply came back the next cycle and could not be confused with anyone else's. With a slow slave several requests and replies overlap in time, so pending_rd_master_q is needed to send each reply to the master that actually asked. It assumes one outstanding read per master, which holds because each slave is blocking (it drops ready during a miss). The core pairs this with c2a_load_pending_q, which tracks the same outstanding load on the CPU side.

9 Atomic unit (amo_unit)

Source: ***design/core/amo_unit/src/.***

The atomic unit runs the RV32A instructions: `lr.w` (load-reserved), `sc.w` (store-conditional), and the `amo*.w` read-modify-write operations (swap, add, and, or, xor, min, max). Each one needs more than one bus access, so the unit uses a small FSM and stalls the IE stage while it works.

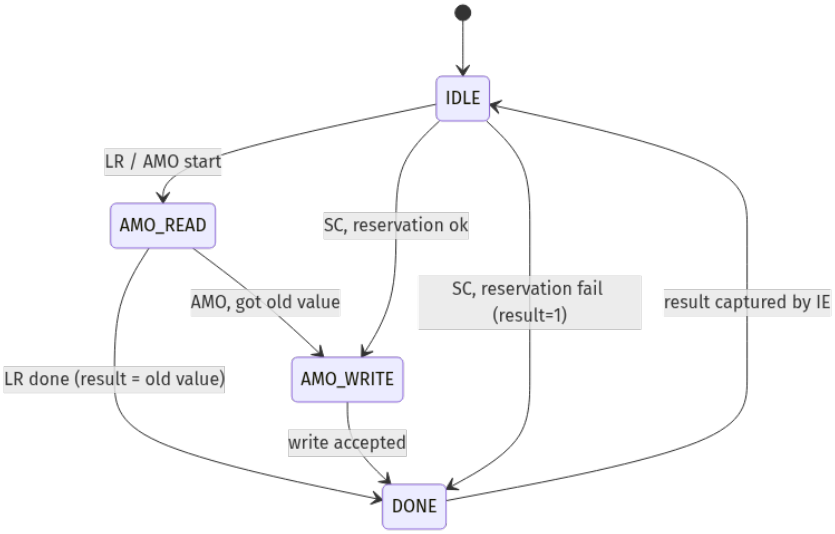


Figure 8: The AMO FSM. A load-reserved or AMO reads first; an AMO then writes the modified value; a store-conditional writes only if its reservation is still valid. `result_o` returns the old value (LR/AMO) or success/fail (SC).

9.1 Ports and state

Inputs from IE: `amo_op_i` (which operation), `addr_i` (the address, from the ALU), `rs2_i` (the second operand), and `flush_i` (a trap is taken). Bus side: `dbus_read_o`/`dbus_write_o`/`dbus_addr_o`/ `dbus_writedata_o` out, `dbus_readdata_i`/`dbus_stall_i` in. Outputs to the core: `result_o` (value for `rd`), `stall_o` (stall IE), `active_o` (the unit is driving the bus), and `pending_o` (an AMO is waiting at IE but has not started yet).

State: `state` (`IDLE`/`AMO_READ`/`AMO_WRITE`/`DONE`), `addr_q`/`rs2_q`/`op_q` (latched request), `read_data_q` (the value read), and the reservation `resv_valid`/`resv_addr` for LR/SC.

9.2 How it runs

On `lr.w` or any `amo*.w` the FSM goes to `AMO_READ` and reads the word. For `lr.w` it returns the value and sets the reservation (`resv_valid`, `resv_addr`). For an `amo*.w` it keeps the old value in `read_data_q`, computes "old OP `rs2`", and goes to `AMO_WRITE` to store it back; `result_o` is the old value. On `sc.w` the FSM first checks the reservation: if `resv_valid` and `resv_addr == addr_i` it writes and returns 0 (success); otherwise it writes nothing and returns 1 (fail). Any `sc.w` or `flush_i` clears the reservation.

Latency note. The FSM advances only when the bus is free: each step waits on `~dbus_stall_i`. A slow slave just holds the unit in `AMO_READ` or `AMO_WRITE` longer. `pending_o` matters for traps: if an interrupt arrives while an AMO is still waiting at IE, the trap saves `pc_ie` so the AMO re-runs after `sret` instead of being skipped. The unit also reads the bus reply through the per-master `amo_arb_rvalid` from `dmem_arb`, so a PTW reply can never be mistaken for the AMO's own data.

Part III

Fetch and branch prediction

10 Fetch front-end

Sources: `program_counter.sv`, `redirect_arbiter.sv`, `fetch/` (`fetch_buffer`), `c_ext/` (`compressed_aligner`) and the fetch wiring in `core_top.sv`.

The job of the front-end is simple to say and hard to do: keep giving the decoder the next instruction. It must turn a stream of 32-bit memory words into aligned instructions (some are 16-bit compressed), it must redirect when a branch or trap changes the path, and it must wait when the icache is slow. Most of the stall-tolerance bugs in this project lived here.

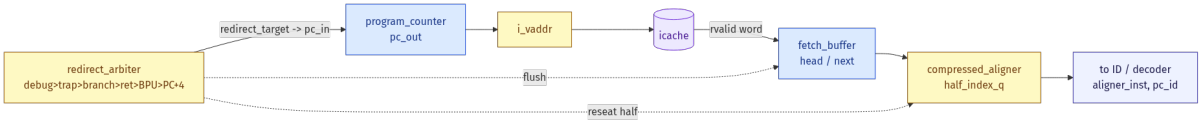


Figure 9: The fetch path. `program_counter` drives `i_vaddr` into the `icache`; returned words go into the `fetch_buffer`; the `compressed_aligner` cuts them into instructions for the decoder. The `redirect_arbiter` picks the next PC and, on a redirect, flushes the buffer and reseats the aligner.

10.1 Program counter and the next-PC choice

`program_counter` holds the fetch address `pc_out`. Each cycle it loads `pc_in` unless `stall_i` is high. `pc_in` comes from the `redirect_arbiter`, which picks one source by priority:

debug (`dpc`) > trap (`handler_addr`) > branch (`branch_target`) > `mret`/`sret` (`mepc`/`sepc`) > BPU prediction > else `pc_out + 4`.

The arbiter outputs `arb_redirect_valid`, `arb_redirect_target`, and `arb_redirect_kind` (`RDR_DEBUG`/`RDR_TRAP`/`RDR_BRANCH`/ `RDR_XRET`/`RDR_BPU`/`RDR_BPU_IF`). A redirect bypasses normal stalls so the new path starts right away.

10.2 The fetch buffer

The address that the producer sends to the icache is `i_vaddr`. Normally it is `pc_out`; it is overridden in special cases (a deferred redirect waiting on the MMU uses `pending_target_q`; the first fetch after reset is held by `first_fetch_pending_q`). When the icache returns a word (`imem_port.rvalid`), the front-end pushes it into the `fetch_buffer`.

The buffer holds a few entries so a 32-bit instruction split across two words can be built from `head` and `next`. Each entry carries the word, its word-aligned `vaddr`, a fault bit and cause (for instruction faults that ride with the word), and the BPU prediction captured at fetch time.

Latency note. The push is gated: `fb_push_raw = imem_port.rvalid & inflight_q & !arb_redirect_flushing & !xret_drop_push`. The `inflight_q` bit means "a fetch we accepted is still waiting for its rvalid." Under variable icache latency a stale rvalid can arrive *after* a redirect already changed the path; `inflight_q` is clear by then, so the stale word is dropped instead of pushed. There is also a `last_pushed_vaddr_q` check that drops a duplicate push when the producer re-issued the same address.

10.3 The compressed aligner

`compressed_aligner` reads `head`/`next` and emits one instruction. It keeps `half_index_q`: 0 means "use the low 16 bits of head", 1 means "use the high 16 bits". It looks at the two opcode bits:

- bits `== 11`: a full 32-bit instruction. If it sits in the high half it straddles into `next`, so the aligner stitches `head[31:16]` with `next[15:0]`.
- bits `!= 11`: a 16-bit compressed instruction. `c_dec` expands it to its 32-bit form.

It outputs `aligner_inst` (the 32-bit instruction), `pc_id` (its address), `aligner_valid`, `is_compressed`, and any `aligner_fault`/`aligner_cause`. The decoder takes it when `aligner_valid & !if_id_stall`.

10.4 Redirect handling

On a redirect the front-end: latches the new target into the PC, raises `fetch_flush` to clear the `fetch_buffer`, and reseats the aligner's `half_index_q` to bit 1 of the target (so a jump to a 16-bit instruction in the high half starts in the right place). One redirect kind is special: `RDR_BPU_IF` (the IF-stage prediction) does *not* flush the buffer, because the branch's own word is still in flight; instead the aligner runs a small "squash" that drops the wrong-path tail of the current word until the predicted target word arrives.

Latency note. Two front-end stall ideas matter under variable latency. `first_fetch_pending_q` holds the reset PC on `i_vaddr` until the icache actually accepts it, so the very first instruction is never lost while the slave is not ready. `redirect_deferred`/`pending_target_q` hold a redirect target when the icache is busy (or the MMU is walking), and replay it when the front-end is free, so a redirect issued during a stall is not dropped.

11 Operand forwarding and data hazards

Sources: `forwarding_unit/` (`forwarding_logic`), and the operand muxes + the `branch_taken_valid` gate in `core_top.sv`.

When one instruction writes a register and the next instruction reads it, the reader needs the new value before it has been written back. *Forwarding* sends the result straight from a later stage to the reader, so the pipeline does not have to stall.

11.1 Where a value can come from

`forwarding_logic` compares the reader's source registers (`rs1_i`, `rs2_i`, and `rs3` for the FPU) against the destination registers of the two instructions ahead of it: the one in IMEM (`rd_mem_i`) and the one in IWB (`rd_wb_i`). It outputs one select per operand (`forwarda_id`/`forwardb_id`/`forwardc_id`):

- match in IMEM and it writes a register → `FORWARD_IMEM` (use `exec_result_imem`); this is the freshest value;
- else match in IWB → `FORWARD_IWB` (use `exec_result_iwb`);
- else `NO_FORWARD` (read the register file).

`x0` never forwards (it is always zero). The IE stage uses these selects to build `opA_forwarded_data`/`opB_forwarded_data`/`opC_forwarded_data`.

11.2 The load-use hazard

A load is special. Its value is not the ALU result; it comes back from memory as `readdata_imem`. So when the very next instruction needs a load's result, the data may not be ready yet. With single-cycle memory the load's data arrives exactly when the next instruction needs it, so a plain forward works. With variable latency it does not.

Latency note. This is the single most important hazard for stall-tolerance. When a `jalr` or branch right after a load uses the load's value, it forwards `readdata_imem`. While the load is still waiting for its data, that value is stale, so the branch/jump would compute the wrong target and redirect to garbage. The fix is `branch_taken_valid = bpu_mispredict & ~iwb_stall`: while `iwb_stall` is high (a CPU load is accepted but its `rvalid` has not come back), the branch/JALR does not resolve. The IE stage is frozen anyway, so the redirect simply fires one cycle later, when the load data is valid. A real `ecall`/`pmp`/`vm` test failure was exactly this: `lw` then `jalr` jumped to `0x11111110` (a stale register) and the core ran off into memory.

12 Branch prediction (bpu)

Sources: `design/core/bpu/src/{bpu,bht,btb,ras}.sv` *and its wiring in* `core_top.sv`.

A branch is a problem for a pipeline. The core does not know if a branch is taken until the IE stage. Without help, it must wait. The branch prediction unit (BPU) guesses the branch early, so the pipeline can keep fetching. If the guess is wrong, the core fixes it later and throws away the wrong path.

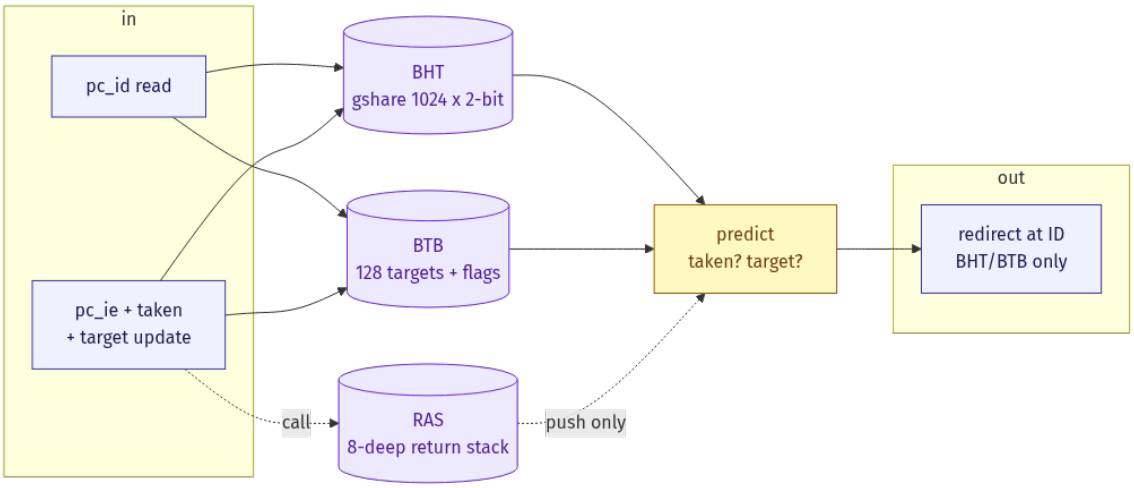


Figure 10: The BPU. It is read at the ID stage to guess the branch, and updated at the IE stage when the real result is known. Three tables work together: the BHT guesses taken or not-taken, the BTB holds the target, the RAS predicts returns.

12.1 The three tables

BHT (branch history table). This guesses the *direction*: taken or not-taken. It has 1024 entries. Each entry is a 2-bit counter (strong/weak, taken/not-taken). It uses "gshare": the index is the PC mixed (XOR) with an 8-bit record of the last 8 branch results. This lets the same branch get a different guess depending on recent history.

BTB (branch target buffer). This holds the *target* address. It has 128 entries, direct-mapped, with a tag to confirm a hit. Each entry also says if the branch is unconditional (a jump, always taken) and if it is compressed (16-bit). A jump is predicted taken without asking the BHT.

RAS (return address stack). This predicts function returns (`jalr` back to the caller). It is an 8-deep stack. A call pushes the return address; a return would pop it.

12.2 When it runs

The BPU is *read* at the ID stage using `pc_id`. If the BTB hits and the branch is predicted taken, the core redirects fetch to the target. The BPU is *updated* at the IE stage, where the real branch result is known: the BHT counter moves toward the result, and the BTB learns the target.

12.3 Spotting a wrong guess

At the IE stage the core compares the real result with the guess. A guess is wrong if the direction is wrong, or if the direction was right but the target was wrong. On a wrong guess the core redirects to the correct PC and flushes the wrong-path instructions in IF/ID.

12.4 What is on and off today

Not every BPU path is enabled. The code keeps some off on purpose:

path	status	why
BHT + BTB at ID	**on**	the main predictor; does most of the work
RAS push	**on**	keeps the return stack correct; no redirect, so safe
RAS pop redirect	**off**	it broke Linux boot: the pop-redirect disturbed ITLB/PTW state and made the kernel's first `satp` write page-fault
IF-stage predict	**off**	costs about 4% on Dhrystone, but ID-stage prediction already gives most of the gain; kept off for simpler debug and Linux

The RAS still pushes and pops internally (so its state stays correct), it just does not drive a fetch redirect.

Part IV

Decode and execute

13 Decode and execute datapath

Sources: *design/core/c_ext/src/c_dec.sv*, *design/core/control_path/src/decoder.sv*, *design/core/imm_gen/src/imm_gen.sv*, *design/core/regfile/src/reg_file.sv*, *design/core/alu/src/alu.sv* (+ *divider.sv*, *clmul.sv*, *zba_zbb.sv*, *zbs.sv*).

13.1 Purpose

This is the middle of the pipe: take a raw instruction word, turn it into control fields and operands, then compute a result. Decode is one combinational stage (the ID stage). Execute is the ALU in the IE stage. Most operations finish in the same cycle; only divide/remainder and the optional FPU take more than one cycle, and they are the only parts of this datapath that can stall the pipeline.

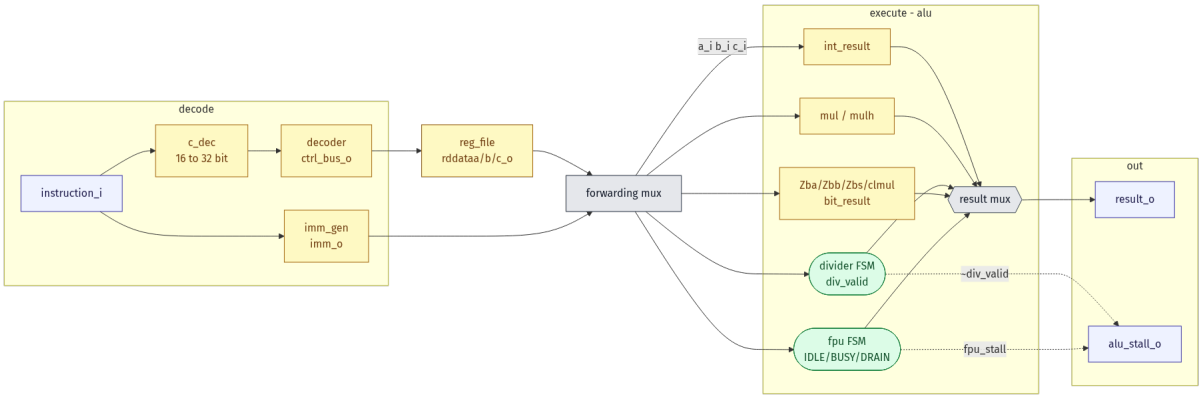


Figure 11: Decode and execute datapath. The instruction word enters on the left, is expanded (if compressed) and decoded into `ctrl_bus_o` plus an immediate, operands are read and forwarded, and the ALU produces `result_o`. Only the two FSM blocks (green)—the divider and the FPU—can raise `alu_stall_o`.

13.2 Decode

`c_dec` (compressed expander). A 16-bit RVC instruction is first expanded to its 32-bit equivalent. `c_dec` is pure combinational: it keys on `{ins_16[15:13], ins_16[1:0]}` (the funct3 field plus the 2-bit quadrant) and builds the matching 32-bit word `ins_32`. Compressed register fields name `x8`–`x15`, so the 3-bit field is widened by prefixing `2'b01`. Immediates are unpacked and re-shuffled into the I/B/J layout the base ISA uses. An unknown 16-bit pattern produces `ins_32 = 0`, which the main decoder treats as illegal.

`decoder`. The 32-bit word (native, or expanded by `c_dec`) is decoded combinationally into one packed struct, `ctrl_bus_o` (type `ctrl_bus_e`). Every field is given a safe default at the top of the block, then overridden by the matching opcode case. The important fields:

field	meaning
imm_sel	immediate format for imm_gen (I/S/B/U/J/CSR)
alu_op	base integer op (ADD, SUB, SLL, SLT…, or NO_ALU_OP)
mul_op	M-extension op (MUL, MULH…, DIV, REM…, or NO_MUL_OP)
bit_op	Zba/Zbb/Zbs/Zbc op (or NO_BIT_OP)
float_op	FP op (or NO_FP_OP); roundmode = DYN reads frm
operand_a/b/c	operand source: REGISTER, IMM, PC, NO_OPERAND
rs1_int / rs2_int / rd_int	integer register addresses (plus float variants)
mem_op / load_store_width / mem_unsigned	memory op kind, size, signedness
wb_sel	write-back source: EXEC, MEMORY, PC_WB, NO_WB
csr_op / amo_op	CSR access kind / atomic op kind
ecall, ebreak, mret, sret, sfence_vma, fence_i	system-instruction tags
wb_event / wb_cause / wb_tval / instr_word	trap tagging (see Traps)

The op-class fields are mutually exclusive by construction: a plain add sets alu_op=ADD and leaves mul_op/bit_op/float_op at their NO_* default. That "exactly one class is active" invariant is what the ALU result mux relies on. An opcode that matches no case keeps all defaults (all NO_*); the privilege unit turns that into an illegal-instruction trap.

imm_gen. A small combinational mux selects on imm_sel_i and sign-extends the right instruction bits into imm_o: I-, S-, B-, U-, J-format, plus a zero-extended 5-bit CSR_imm. U-format is the only one not sign-extended (it fills the low 12 bits with 0).

13.3 Operands and the register file

reg_file. 32 registers, combinational reads, one synchronous write. The integer file is parameterised ZERO_REG=1: a read of x0 returns 0 and a write to x0 is suppressed. The write is gated by stall_i so a held instruction does not write twice. Two read ports (rddataa_o, rddatab_o) feed rs1/rs2; the third port (rddatac_o) is used only by the FP file (ZERO_REG=0, no x0 special case) for the rs3 of fused multiply-add.

Raw register data is not used directly. It first passes through the forwarding mux, which substitutes a younger in-flight result when a later stage is about to write a register this instruction reads. The selected operand_a/b/c (register, the immediate, or the PC, per the decoder) becomes a_i/b_i/c_i into the ALU. Forwarding and the load-use interlock are covered in their own section.

13.4 Execute (the ALU)

The ALU computes four kinds of result in parallel and selects one. The select is a priority mux on the same NO_* sentinels the decoder set (alu.sv:330):

active field	result source
`alu_op` ≠ `NO_ALU_OP`	`int_result` (add/sub/shift/compare/logic; `PASS`=`b_i` for LUI/AUIPC)
`mul_op` ≠ `NO_MUL_OP`	`mul_result` (combinational MUL/MULH, or the divider output)
`bit_op` ≠ `NO_BIT_OP`	`bit_result` (Zba/Zbb/Zbs combinational, or `clmul` for Zbc)
`float_op` ≠ `NO_FP_OP`	`fpu_result` (only when `FPU` is defined)

Multiply (`MUL`/`MULH`/`MULHU`/`MULHSU`) is a single-cycle `$signed` product, sliced `[31:0]` or `[63:32]`. The bit-manip units (`zba_zbb`, `zbs`, `clmul`) are all combinational; the ALU just routes their outputs through a mux on `bit_op_i`.

Divider. `DIV`/`REM`/`DIVU`/`REMU` assert `div_start`, which kicks the restoring divider (`divider.sv`). It does one quotient bit per cycle. A leading-zero count sets the iteration count `init_n` to the number of significant dividend bits (minimum 3), so small operands finish fast. The special cases—divide-by-zero and signed overflow—are detected up front and forced to the RISC-V-defined results. `valid_o` (= `n==0`) signals completion as `div_valid`.

FPU (optional, `ifdef FPU`). A 3-state handshake FSM (`FP_IDLE` → `FP_BUSY` → `FP_IDLE`, with a `FP_DRAIN` state so a flushed-but-launched op still consumes its result) drives the external `fp_top` core. Bit-move ops (`FMVXW`/`FMVWX`) set `fpu_bypass` and skip the FSM entirely. 32-bit operands are NaN-boxed to 64 bits on the way in.

13.5 Where the pipeline stall comes from

Latency note. This datapath has exactly one stall output, and it is born here, not imported. `alu.sv:344`:

$$\texttt{alu\_stall\_o} = (\texttt{div\_start} \,\&\, \neg\texttt{div\_valid}) \;\vee\; \texttt{fpu\_stall}.$$

A divide holds the IE stage until `div_valid`; an FP op holds it until the FSM returns to `FP_IDLE`. In `core_top`, `alu_inst.stall_i` is tied to `1'b0`: the ALU's own latches are never frozen, because the divider *is* the thing generating the stall and must keep iterating while the rest of the pipe waits on it. `alu_stall_o` leaves as `alu_stall` and folds into `ie_stall` in the hazard unit; `ie_stall` then propagates up to `if_id_stall`, freezing the front-end until the multi-cycle op retires. Unlike the load path, this multi-cycle behaviour was *always* present—it is internal to the core and does not depend on memory latency—so it needed no change for the variable-latency work. The contrast is the point: the ALU was built to stall on its own long ops, while the load path originally assumed memory never stalled at all.

Part V
System and software

14 SoC overview

Sources: *design/soc_top/src/soc_top.sv, design/interconnect/src/addr_decoder.sv, design/interconnect/src/periph_bridge.sv, design/common/mem_map.svh.*

14.1 Top-level structure

`soc_top` ties the core to memory and the peripherals. The core exposes two ports, both on the latency-tolerant `mem_bus` interface: `imem_bus` (fetch) and `dmem_bus` (load/store). The instruction side goes through `icache` into port A of a dual-port RAM (`ram_dp`, or `ram_dp_delayed` for variable-latency tests). The data side goes through `addr_decoder`: requests for the RAM range go to `dcache` and port B of the same RAM; everything else goes through `periph_bridge` to the peripherals. Interrupts come back from `clint` (timer/software) and `plic` (external).

14.2 Memory map

All bases live in `mem_map.svh`. The decode splits cleanly: RAM is high memory, the `0x1000_xxxx` block is the peripheral window (selected by `addr[19:16]`), and the standard RISC-V cores sit at their architectural addresses.

region	base	notes
RAM	`0x8000_0000`	main memory (`RAM_BASE`; size from `RAM_SIZE_BYTES`)
boot ROM	`0x0000_1000`	reset vector lands here (sim: `bootmem`)
UART	`0x1000_0000`	`addr[19:16]`=0; sifive,uart0
SPI	`0x1001_0000`	`addr[19:16]`=1; sifive,spi0
I2C	`0x1002_0000`	`addr[19:16]`=2; opencores i2c
GPIO	`0x1003_0000`	`addr[19:16]`=3; sifive,gpio0
PWM	`0x1004_0000`	`addr[19:16]`=4; sifive,pwm0
TIMER	`0x1005_0000`	`addr[19:16]`=5
CRC	`0x1006_0000`	`addr[19:16]`=6; accelerator
CLINT	`0x0200_0000`	MSIP / MTIMECMP / MTIME
PLIC	`0x0C00_0000`	priority / pending / enable / claim

14.3 Address decode and response mux

`addr_decoder` is pure combinational: it compares `addr_i` against each base/range and drives a one-hot of selects (`ram_sel_o`, `uart_sel_o`, `clint_sel_o`, ...). The peripheral selects share the `0x100_xxxxx` test plus a 4-bit `addr[19:16]` compare. On the return path, `periph_bridge` latches which peripheral was read and, one cycle later, muxes its `readdata` onto `rdata_o` with `rvalid_o`—peripherals are single-cycle and always ready.

14.4 Variable-latency read routing: rd_inflight_q

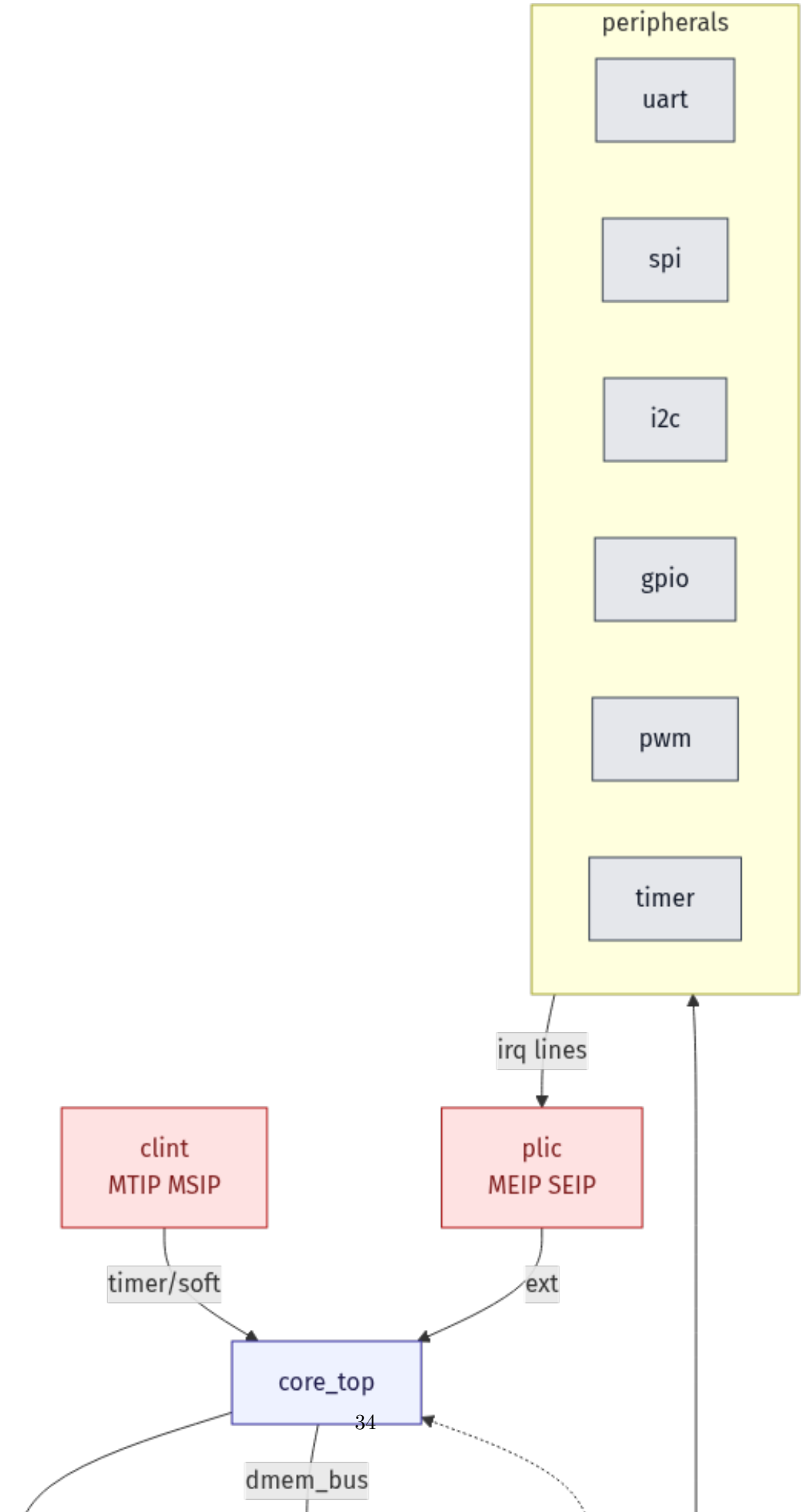
Latency note. The d-bus has two possible responders (dcache for RAM, periph_bridge for everything else), and they can answer with different latencies. The hazard is timing: once a read is *accepted*, the LSU moves dmem_bus.addr on to the next access, so ram_sel no longer reflects the *outstanding* read. A response muxed on live ram_sel would then be routed to the wrong responder—under always-ready RAM the response came back the same cycle so this never showed, but under variable latency the load result was silently dropped. The fix (soc_top.sv:303--322) captures the target at accept time. d_read_accept = dmem_bus.req & ~dmem_bus.we & dmem_bus.ready; on that edge rd_inflight_q←1 and rd_to_ram_q←ram_sel (a snapshot). The response is then routed by the *snapshot*, not the live select: dmem_bus.rdata = rd_to_ram_q ? dc_cpu_rdata : periph_rdata, and rvalid is gated by rd_inflight_q. rd_inflight_q clears when the response is consumed. This is the SoC-level twin of dmem_arb's per-master pending_rd_master_q: both exist to make sure a delayed read result reaches exactly the requester that is waiting for it. The design is single-outstanding on the d-bus, so one snapshot flop is enough.

14.5 Reset and boot

reset_i fans out to the core, caches, RAM, and every peripheral; the debug module can additionally assert ndmreset to reset the core over JTAG. On reset the program counter loads its DEFAULT vector (0x0000_1000 for the boot ROM build, or 0x8000_0000 for a RAM-resident image), and the core begins fetching there through imem_bus → icache.

14.6 Interrupts

Two cores deliver interrupts. clint provides the machine timer (MTIP, from mtime ≥ mtimecmp) and software (MSIP) interrupts straight to the core. plic aggregates the peripheral interrupt lines (UART rx/tx, SPI, I2C, GPIO) and presents two contexts: M-mode (ext_interrupt → MEIP) and S-mode (s_ext_interrupt → SEIP), each with its own enable, threshold, and claim/complete registers. This split (CLINT for timer/IPI, PLIC for device IRQs with priority and per-mode contexts) is what lets the Linux driver model bind to the SoC directly, which the peripheral standardisation work targets.


peripherals
uart
spi
i2c
gpio
pwm
timer
irq lines
clint
MTIP MSIP
plic
MEIP SEIP
timer/soft
ext
core_top
dmem_bus

15 Linux boot: software and hardware

Sources: `flows/simulation/run_linux.sh`, `software/linux/` *(OpenSBI platform, kernel, device tree),* `design/common/mem_map.json`, `design/uncore/{clint,plic,uart}`.

This section follows a full Linux boot. It shows what the software does and what hardware the core must provide at each step. Linux uses the "linux" memory profile: 128 MB of RAM at `0x80000000`.

15.1 Phase 1 — reset (M-mode)

After reset the core is in M-mode. The PC starts at `0x80000000`, the start of RAM. The single image (`fw_payload`) is already loaded there: it contains OpenSBI, the Linux kernel, and the device tree together. The first instructions run with no translation (M-mode does not translate).

Hardware used: reset to a known PC, instruction fetch from RAM.

15.2 Phase 2 — OpenSBI (M-mode firmware)

OpenSBI is the M-mode firmware. It sets up the machine and then hands control to Linux in S-mode. Its main jobs:

- set up the UART (console) at `0x10000000` for early printing;
- set up the CLINT timer/software interrupts at `0x02000000` and the PLIC external-interrupt controller at `0x0C000000`;
- set PMP regions so S-mode can access memory;
- set `menvcfg`: turn on **STCE** (the Sstc S-mode timer) and **ADUE** (hardware A/D bit updates in the page walker). Linux needs ADUE, or the page walker would fault every time it has to set a dirty bit;
- delegate most traps and interrupts to S-mode, set `mtvec`, then `mret` into the kernel in S-mode.

Hardware used: M-mode traps, CLINT, PLIC, UART, PMP, `menvcfg`.

15.3 Phase 3 — Linux kernel (S-mode)

The kernel starts in `head.S`. It turns on paging by writing `satp` with Sv32 mode and the root page-table address. From now on every address is a virtual address and goes through the MMU. It sets `stvec` (the S-mode trap vector) so the kernel can handle its own traps and page faults. It programs the timer with the Sstc `stimecmp` CSR, so timer ticks come straight to S-mode. Then it brings up drivers and mounts the root filesystem.

Hardware used: S-mode traps, the Sv32 page walker and TLBs, page-fault traps (causes 12/13/15), the Sstc timer, atomic instructions (AMO: LR/SC for kernel locks), the PLIC S-mode context, and the UART.

15.4 Phase 4 — userspace (U-mode)

The kernel starts `/init` (busybox) in U-mode. User programs run with the lowest privilege. Any system call (`ecall`) or page fault traps into the kernel in S-mode, which serves it and

returns with `sret`.

Hardware used: U-mode, `ecall` traps, page faults, the full MMU.

15.5 Memory map (linux profile)

range	device
`0x02000000`	CLINT (`mtime`, `mtimecmp`, `msip`)
`0x0C000000`	PLIC (external-interrupt controller)
`0x10000000`	UART (console)
`0x10010000` +	SPI, I2C, GPIO, PWM, CRC
`0x80000000`	RAM, 128 MB (firmware + kernel + rootfs)

15.6 CLINT and PLIC in short

The **CLINT** holds `mtime` (a free-running counter) and `mtimecmp`. It raises the timer interrupt when `mtime` $\geq$ `mtimecmp`, and raises a software interrupt when `msip` is set. The **PLIC** collects external interrupts from the peripherals (UART, SPI, I2C, GPIO) and routes them to a "context". Context 0 is M-mode, context 1 is S-mode, so Linux takes external interrupts in S-mode directly.

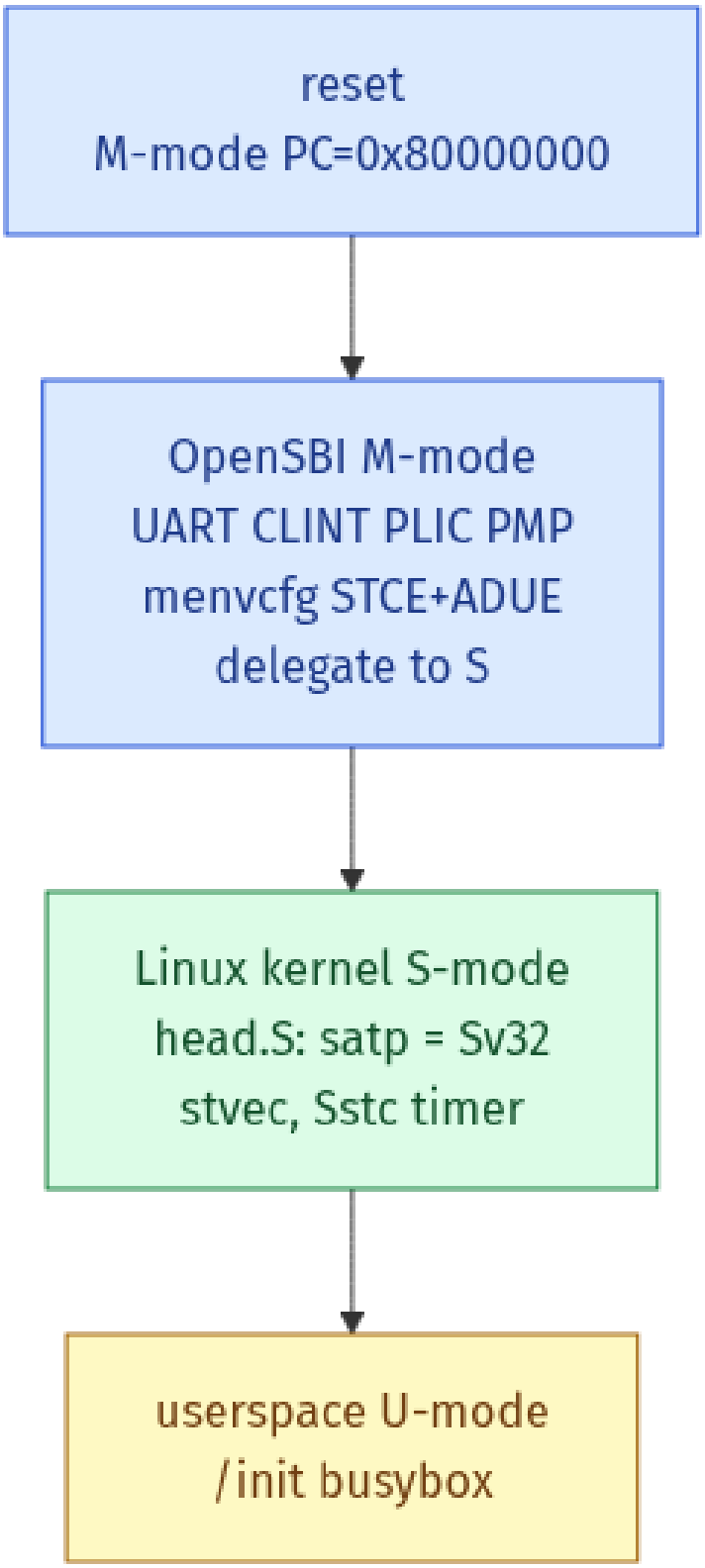


Figure 13: The four boot phases. Privilege drops from M to S to U. Each box lists the main work of that phase.